# Conteneuriser l'application

## Un clone neuf, une commande, le même résultat partout

**Séquence**
SQ01

**Partie**
P03

**Niveau**
Mastère ESI

# Objectifs

- Distinguer image, conteneur, registre et volume
- Écrire le `Dockerfile` de `voyages-scolaires` et expliquer chaque instruction
- Démarrer l'application et sa base avec Docker Compose
- Expliquer ce qui rend un démarrage **reproductible**

**Prérequis :** Docker Desktop (ou Docker Engine) installé ; `main` à jour.

ATTENTION

# Le problème

Le clone neuf de tout à l'heure a demandé une dizaine de commandes.

Il fallait **la bonne version de PHP**, les bonnes extensions, `composer install`, un `.env`, une clé, une base, les migrations…

**Comment garantir que l'appli démarre pareil sur chaque poste, et en production ?**

# Machine virtuelle ou conteneur ?

| | Machine virtuelle | Conteneur |
|---|---|---|
| Système | Un OS invité complet | Partage le noyau de l'hôte |
| Démarrage | Minutes | Secondes |
| Taille | Plusieurs Go | Quelques dizaines à centaines de Mo |
| Isolation | Forte | Processus isolés |
| Usage typique | Héberger des systèmes | Livrer une application et ses dépendances |

# Quatre mots à ne pas confondre

| Terme | Ce que c'est | Analogie |
|---|---|---|
| **Image** | Modèle en lecture seule : système, dépendances, code | Une classe |
| **Conteneur** | Une instance de l'image qui s'exécute | Un objet |
| **Registre** | Le dépôt où l'on publie et récupère les images | Packagist, pour les images |
| **Volume** | Un stockage qui survit à la suppression du conteneur | Le disque de la base |

# Premiers gestes

```
docker run --rm hello-world   # télécharger, lancer, supprimer
docker images                 # les images présentes
docker ps -a                  # les conteneurs, même arrêtés
docker logs <conteneur>       # ce qu'il a écrit
docker exec -it <conteneur> sh  # entrer dedans
```

Un conteneur arrêté n'est pas supprimé : `--rm` le supprime en sortant.

# La pile de Voyages scolaires

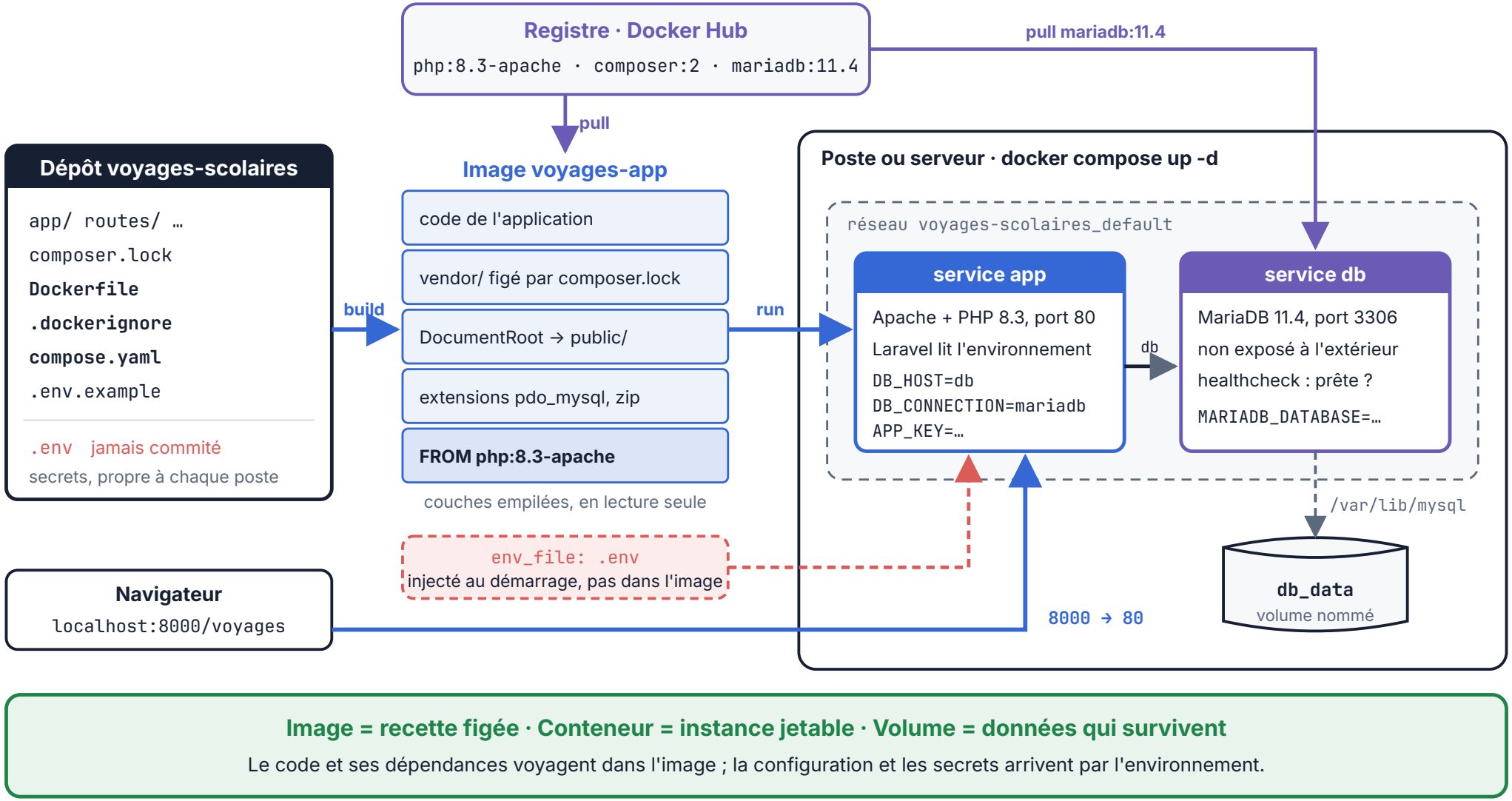



**Image = recette figée · Conteneur = instance jetable · Volume = données qui survivent**

Le code et ses dépendances voyagent dans l'image ; la configuration et les secrets arrivent par l'environnement.

# Le Dockerfile, 1/2 : le système

```
FROM php:8.3-apache

RUN apt-get update \
    && apt-get install -y --no-install-recommends \
       git unzip libzip-dev \
    && docker-php-ext-install pdo_mysql zip \
    && a2enmod rewrite \
    && rm -rf /var/lib/apt/lists/*

ENV APACHE_DOCUMENT_ROOT=/var/www/html/public
RUN sed -ri -e 's!/var/www/html!${APACHE_DOCUMENT_ROOT}!g' \
    /etc/apache2/sites-available/*.conf
```

Une version de PHP **écrite**, plus jamais « celle installée sur le poste ».

# Le Dockerfile, 2/2 : l'application

```
COPY --from=composer:2 /usr/bin/composer /usr/bin/composer
WORKDIR /var/www/html

COPY composer.json composer.lock ./
RUN composer install --no-dev --no-scripts --no-autoloader

COPY . .
RUN composer dump-autoload --optimize \
    && chown -R www-data:www-data storage bootstrap/cache
```

`composer.lock` d'abord : tant qu'il ne change pas, Docker **réutilise** la couche `vendor/`.

# Ce qui n’entre pas dans l’image

```
# .dockerignore
.git
.env
vendor
node_modules
database/*.sqlite
```

- `.env` : les secrets n’ont rien à faire dans une image publiée
- `vendor` : reconstruit **dans** l’image, depuis `composer.lock`
- Moins de fichiers envoyés, construction plus rapide

# compose.yaml : l'application

```yaml
services:
  app:
    build: .
    ports:
      - "8000:80"
    env_file: .env
    environment:
      DB_CONNECTION: mariadb
      DB_HOST: db
    depends_on:
      db:
        condition: service_healthy
```

`db` est le **nom du service** : Compose le résout sur le réseau interne.

# compose.yaml : la base

```yaml
  db:
    image: mariadb:11.4
    environment:
      MARIADB_DATABASE: ${DB_DATABASE}
      MARIADB_USER: ${DB_USERNAME}
      MARIADB_PASSWORD: ${DB_PASSWORD}
      MARIADB_RANDOM_ROOT_PASSWORD: "1"
    volumes:
      - db_data:/var/lib/mysql
    healthcheck:
      test: [CMD, healthcheck.sh, --connect,
             --innodb_initialized]
```

En fin de fichier : `volumes: { db_data: }`.

# La configuration vient de l'environnement

- Le **même** `.env` sert à Laravel et à Compose ( `${DB_DATABASE}` )
- Une variable d'environnement **l'emporte** sur le fichier `.env` lu par Laravel
- `.env.example` est commité, sans secrets ; `.env` reste sur chaque poste
- La base n'est pas exposée : seul `app` l'atteint, par le réseau interne

> *Une image, plusieurs environnements : seule la configuration change.*

# Démarrer la pile

```
docker compose up -d --build
docker compose ps
docker compose exec app php artisan migrate --seed
docker compose logs -f app
```

Ouvrir `http://localhost:8000/voyages`, puis `/api/voyages`.

`docker compose down` arrête tout ; `down -v` supprime **aussi** le volume, donc les données.

# Pourquoi c'est reproductible

- **Versions figées** : `php:8.3-apache`, `mariadb:11.4`, jamais `latest`
- **Dépendances verrouillées** : `composer.lock` commité, `composer install` dans l'image
- **Rien du poste** dans l'image : `.dockerignore`, configuration par l'environnement
- **Tout est décrit** dans le dépôt : `Dockerfile`, `compose.yaml`, revus par PR

*Deux `docker compose up` sur deux postes : même système, mêmes dépendances, même résultat.*

# Quand ça ne démarre pas

| Symptôme | Premier contrôle |
| --- | --- |
| `port is already allocated` | Un `php artisan serve` tourne encore ? Changer `"8001:80"` |
| Page 500, `No application encryption key` | `APP_KEY` présent dans `.env` ? |
| `Connection refused` vers la base | `DB_HOST=db`, et `docker compose ps` : `db` est-elle *healthy* ? |
| Accès refusé pour l'utilisateur | Identifiants changés après le 1er démarrage : `down -v` |
| `Permission denied` dans `storage/` | Le `chown` du Dockerfile |

ACTIVITÉ

# À trois : conteneuriser par PR

Une branche chacun · revue en cercle · support A4, partie 3

**À produire :** `main` démarre depuis un clone neuf avec `docker compose up`.

1. **API —** `Dockerfile` et `.dockerignore`
2. **Création —** `compose.yaml` et `.env.example` complété
3. **Modification** — section « Démarrer avec Docker » du README, puis test du clone neuf

À VOUS

# Restitution

Clone neuf sur le poste enseignant, devant l'équipe :

1. Combien de commandes, cette fois ?
2. Chacun explique une instruction de son fichier
3. `down`, puis `up` : les voyages sont-ils toujours là ? Et après `down -v` ?

**Qu'est-ce qui pourrait encore différer entre deux postes ?**

À RETENIR

# À retenir

- **Image** = recette figée · **conteneur** = instance jetable · **volume** = données durables
- Versions figées et `composer.lock` : la reproductibilité s'écrit
- Secrets et configuration par l'**environnement**, jamais dans l'image
- Les services se parlent **par leur nom** sur le réseau Compose

Intervention 4 : construire cette image automatiquement à chaque PR.